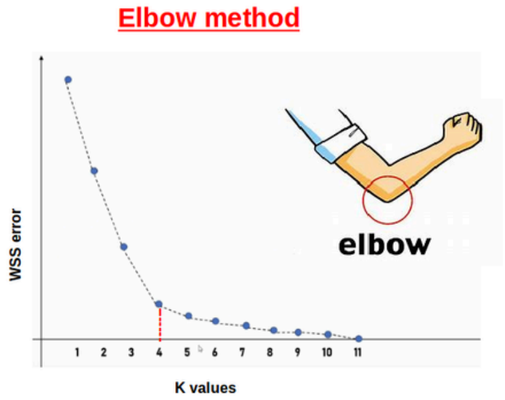
KAZAKH-BRITISH TECHNICAL UNIVERSITY

School of IT and Engineering

MACHINE LEARNING

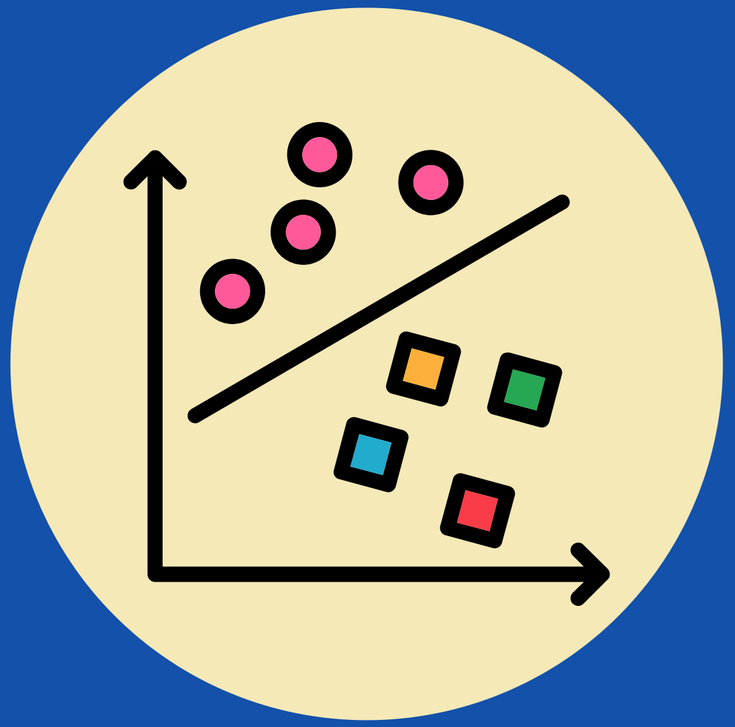
LECTURE #7



Number of credits: 3 (2/0/1)
Course code – CSCI3234
MS Teams team code – **y4ntwlz**

Adilet Yerkin, MS in Engineering, Senior Lecturer
a.yerkin@kbtu.kz

Clustering techniques



Clustering techniques are unsupervised machine learning methods used to group a set of data points into clusters, so that data points within the same cluster are similar to each other, while those in different clusters are dissimilar.

The goal of clustering is to discover structure or patterns in unlabeled data (data without predefined categories or outputs)

Clustering is not about finding groups — it's about discovering insight.

Clustering

They are used when we don't have predefined labels (like "spam" or "not spam") and want to discover structure or patterns within data.

Clustering in Business Strategy
Scenario:
As the Director of Customer Relationships at AllElectronics, you want to divide customers into five groups, assigning each to a manager.

- Customers in each group should be as similar as possible.
- Customers with very different behaviors should belong to different groups.
- Goal: Create targeted marketing campaigns for each group based on shared features.

Challenge:

- There are many customers and numerous attributes describing them.
- Manual grouping is inefficient and costly.
- Group (class) labels are unknown.

Solution - Clustering Technique

Clustering is for discovery, not just grouping.

Clustering is the process of grouping data objects into clusters so that:

- Objects within a cluster are highly similar.
- Objects between clusters are very different.

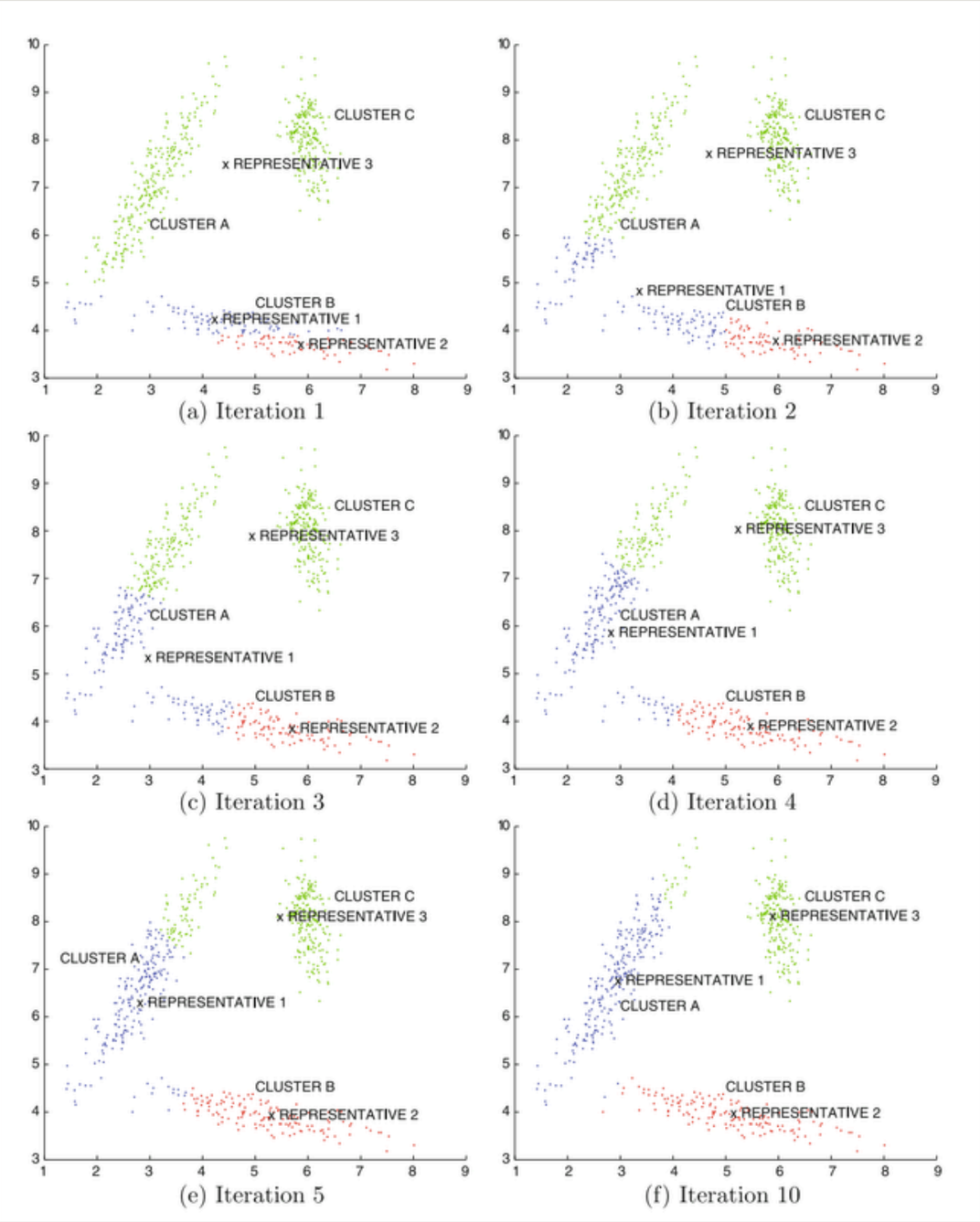


Illustration of k-representative algorithm with random initialization

K-Means Clustering

K-Means is one of the simplest and most widely used clustering algorithms. It partitions data into k clusters, where each data point belongs to the cluster with the nearest mean (called the centroid).

Algorithm Steps:

1. Choose the number of clusters (k).
2. Initialize k centroids randomly in the feature space.
3. Assign each data point to the closest centroid (using Euclidean distance).
4. Update each centroid as the mean of all points assigned to it.
5. Repeat steps 3–4 until the centroids no longer move significantly (convergence).

Strengths:

- Simple and fast for large datasets.
- Works well when clusters are spherical and similar in size.

Limitations:

- Requires predefining k.
- Sensitive to outliers and initial centroid positions.
- Not suitable for non-convex cluster shapes.

Hierarchical Clustering

Hierarchical clustering builds a hierarchy (tree-like structure) of clusters, known as a dendrogram, showing how clusters are merged or split at different similarity levels.

Types:

- Agglomerative (bottom-up): start with each data point as its own cluster and iteratively merge the two closest clusters.
- Divisive (top-down): start with all points in one cluster and recursively divide it.

Linkage Methods: (how distances between clusters are computed)

- Single linkage: minimum distance between points in two clusters.
- Complete linkage: maximum distance between points in two clusters.
- Average linkage: average distance between all points in the clusters.
- Ward's method: minimizes the total within-cluster variance.

Strengths:

- No need to pre-specify number of clusters (k).
- Produces a visual dendrogram for analysis.

Limitations:

- Computationally expensive for large datasets.
- Once clusters are merged or split, the decision cannot be undone.

Hierarchical Clustering

Agglomerative clustering

Agglomerative clustering is the most commonly used hierarchical clustering method.
It starts with many small clusters and gradually merges them into larger clusters.

Main Idea - Start with each data point as its own cluster, then repeatedly merge the closest clusters until only one cluster remains (or until the desired number of clusters is reached).

Steps of Agglomerative Clustering
Step 1 — Start with Individual Clusters
Each data point forms its own cluster.

Example: Dataset with 5 points: A, B, C, D, E
Initial clusters: {A}, {B}, {C}, {D}, {E}

Step 2 — Compute Distance Matrix
Calculate the distance between all clusters.
This distance can be defined using linkage methods:

- Single linkage (minimum distance)
- Complete linkage (maximum distance)
- Average linkage
- Ward’s method

Hierarchical Clustering

Agglomerative clustering

Step 3 — Find the Two Closest Clusters
Identify the clusters with the smallest distance.

Example: distance(A,B) = 2
Merge clusters: {A,B}

Step 4 — Update Distance Matrix
Recalculate distances between the new cluster and other clusters using the chosen linkage method.
Example:
Clusters now: {A,B}, {C}, {D}, {E}
Update distances accordingly.

Step 5 — Repeat the Merging Process
Repeat:

1. Find closest clusters
2. Merge them
3. Update distances

Continue until:

- all points form one cluster
- or
- desired number of clusters is reached

Hierarchical Clustering

Agglomerative clustering

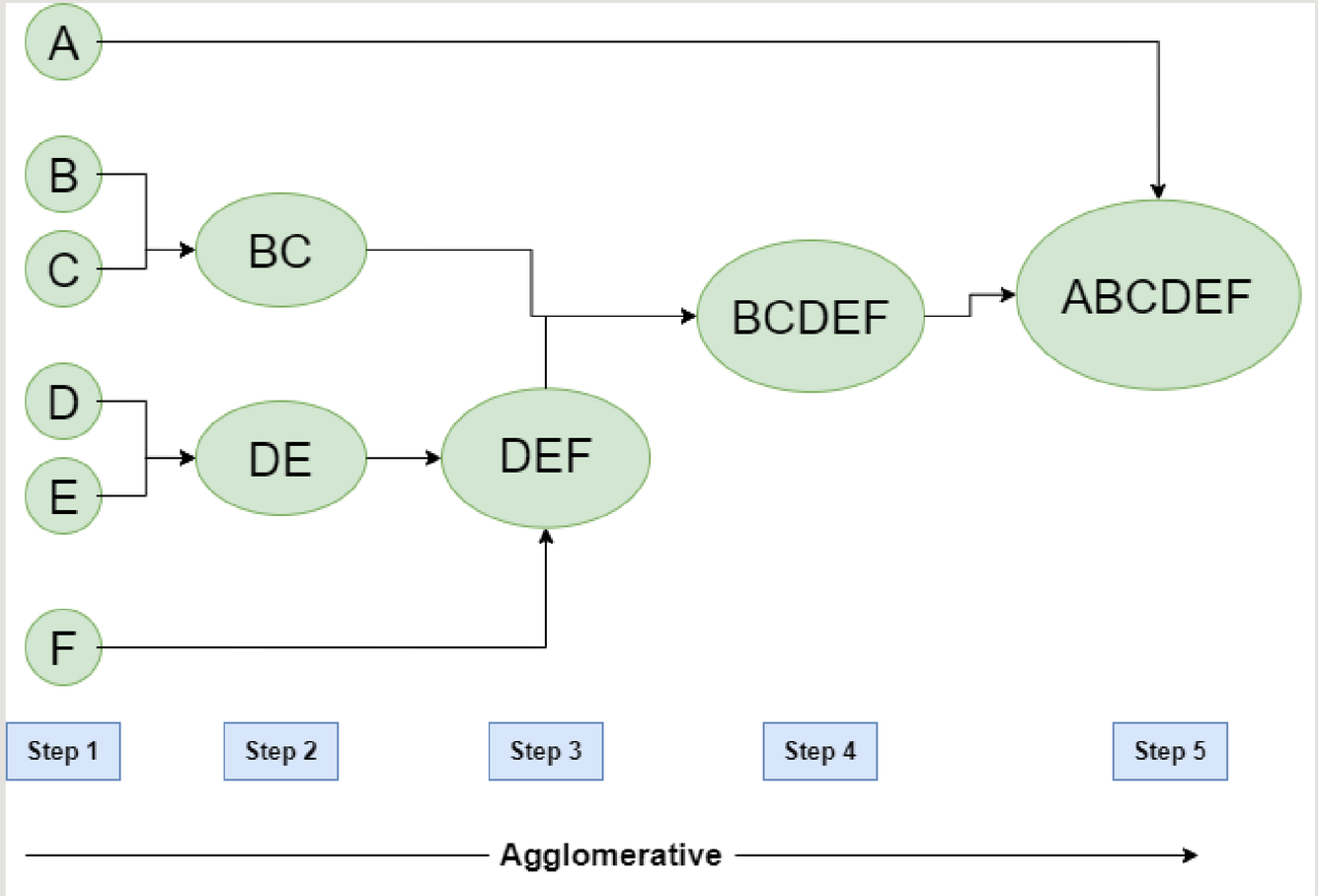
Step 6 — Build the Dendrogram
The algorithm produces a dendrogram, a tree diagram showing how clusters were merged.
This allows analysts to choose a cut level to determine the number of clusters.

Step 1: {A} {B} {C} {D}

Step 2: merge A,B → {AB}

Step 3: merge C,D → {CD}

Step 4: merge AB,CD → {ABCD}



Hierarchical Clustering

Divisive Hierarchical clustering

Divisive clustering works in the opposite direction.
Instead of merging clusters, it starts with one cluster containing all data points and gradually splits it.

Main Idea
Start with all points in one cluster.
Then repeatedly divide clusters into smaller clusters until each point forms its own cluster (or until desired number of clusters is obtained).

Steps of Divisive Clustering
Step 1 — Start with One Cluster
All data points belong to one cluster.
Example: {A, B, C, D, E}

Step 2 — Select Cluster to Split
Choose the cluster with the largest heterogeneity (most variation inside it).

Hierarchical Clustering

Divisive Hierarchical clustering

Step 3 — Split the Cluster
Divide the selected cluster into two smaller clusters. This can be done using:

- K-Means (with k = 2)
- distance-based splitting
- other clustering methods

Example split: {A, B, C, D, E} → {A, B, C} {D, E}

Step 4 — Evaluate Cluster Quality
Check whether the split improves cluster structure. If clusters are still heterogeneous, continue splitting.

Step 5 — Repeat Splitting
Repeat splitting process for each cluster until:

- desired number of clusters is reached or
- each data point becomes its own cluster

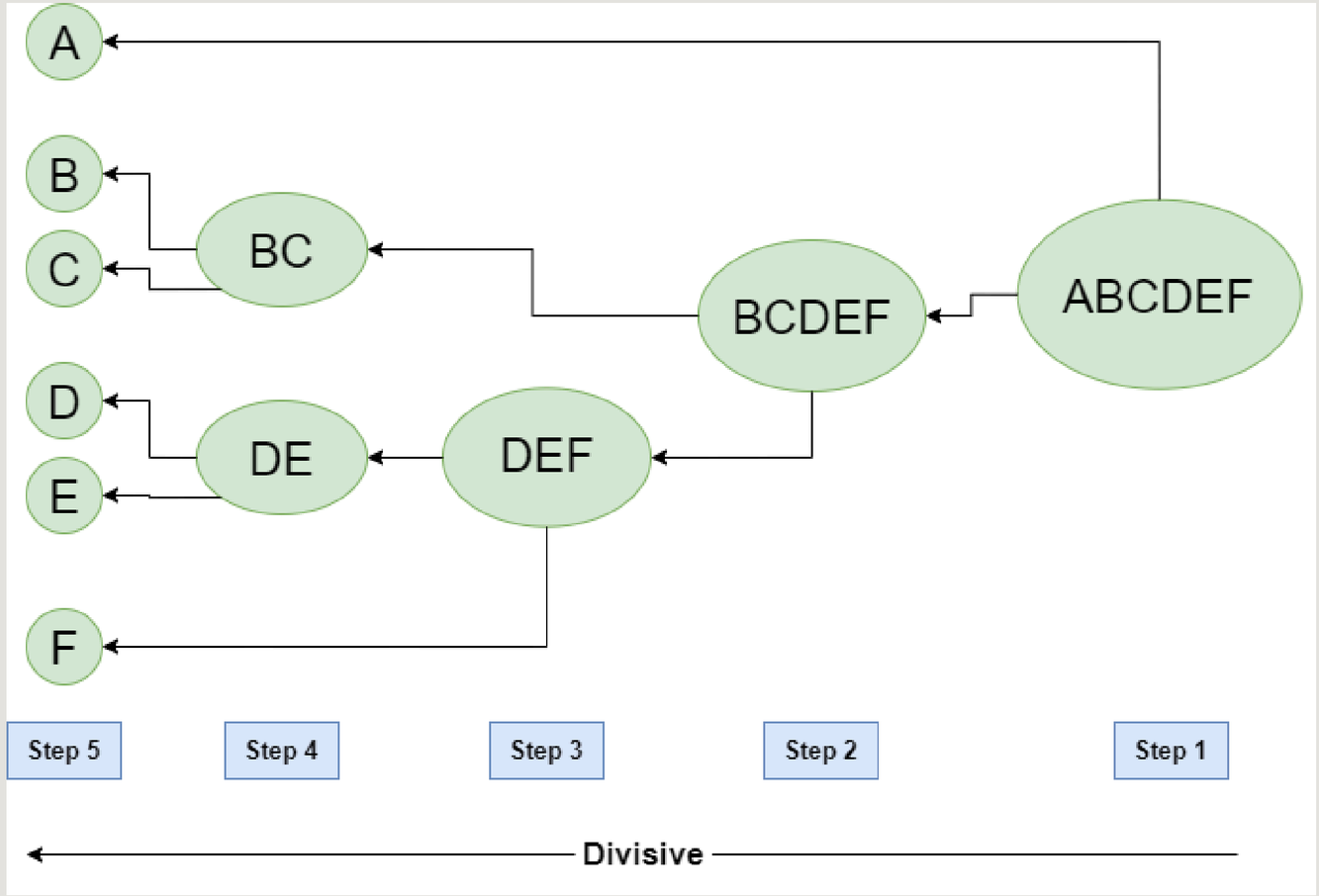
Example:
{A,B,C} → {A,B} {C}
{D,E} → {D} {E}

Hierarchical Clustering

Divisive Hierarchical clustering

Step 6 — Construct the Dendrogram

Divisive clustering also produces a hierarchical tree, but it grows from top to bottom.



Hierarchical Clustering

Agglomerative clustering is much more commonly used because:

- simpler implementation
- lower computational cost
- widely supported in ML libraries (Scikit-learn, R, etc.)

Divisive clustering is conceptually interesting but computationally expensive.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN groups points that are closely packed together (high density) and marks points that lie alone in low-density regions as outliers.

Parameters:

- eps: the maximum distance between two points to be considered neighbors.
- minPts: the minimum number of points required to form a dense region.

Algorithm Steps:

1. For each point, find its neighborhood within distance eps.
2. If it has at least minPts neighbors, it becomes a core point.
3. All points within its neighborhood are assigned to the same cluster.
4. Continue expanding clusters until no more points can be added.
5. Points that are not reachable from any cluster are labeled as noise.

Strengths:

- Detects clusters of arbitrary shape and size.
- Identifies noise and outliers automatically.
- No need to specify number of clusters.

Limitations:

- Choosing appropriate eps and minPts can be difficult.
- Struggles when clusters have varying densities.

Spectral Clustering

Spectral clustering uses graph theory and linear algebra to cluster data that may not be well-separated in Euclidean space.
It represents data as a graph, where nodes are data points and edges connect similar points.
Then it uses eigenvectors of a similarity matrix (Laplacian) to reduce dimensionality before clustering.

Algorithm Steps:

1. Construct a similarity matrix (e.g., using Gaussian kernel).
2. Compute the Laplacian matrix of the graph.
3. Find the first k eigenvectors of the Laplacian.
4. Apply K-Means (or similar method) on these new representations.

Strengths:

- Handles complex, non-convex cluster shapes.
- Effective for data with manifold structures or connected regions.

Limitations:

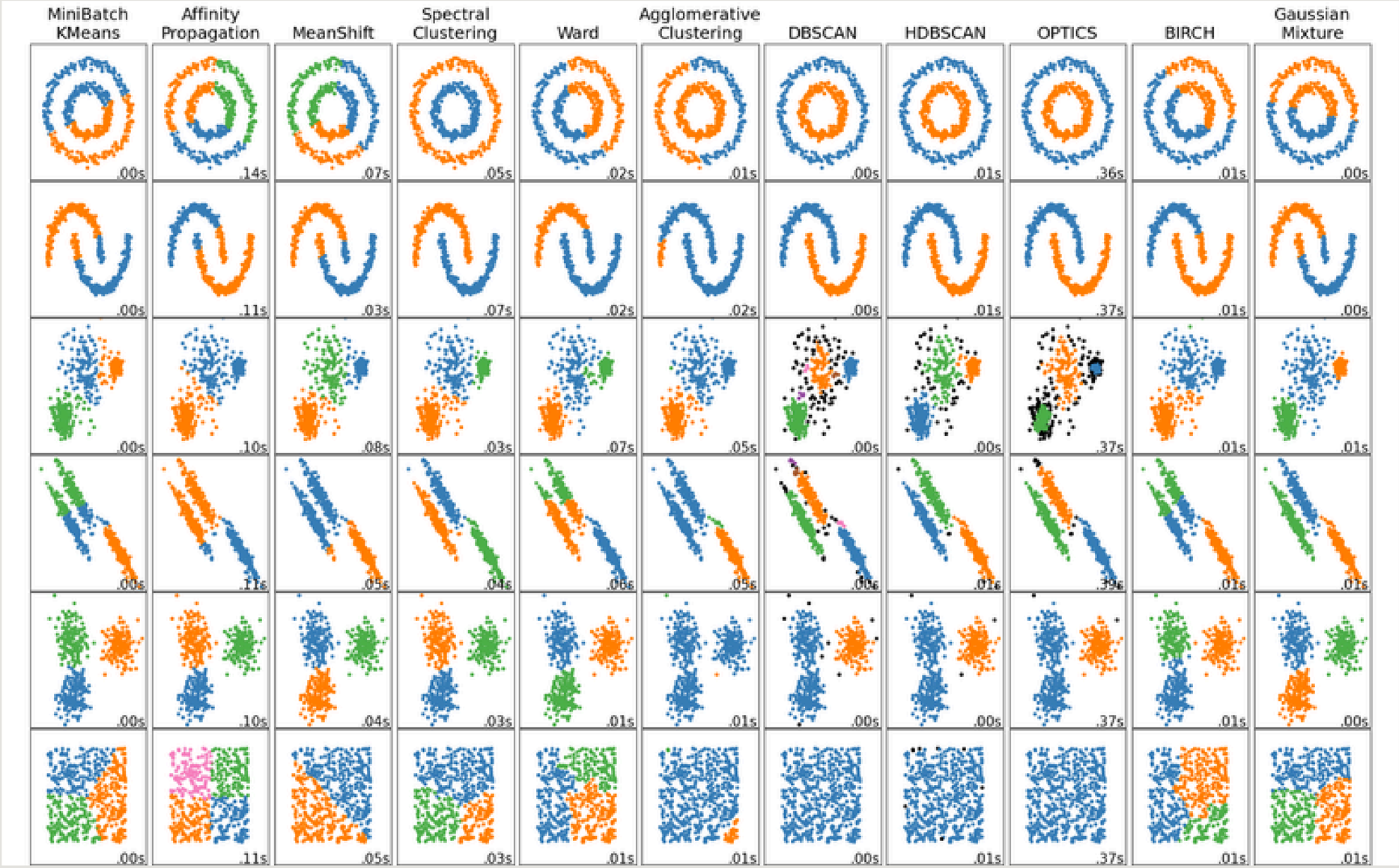
- Computationally expensive for large datasets.
- Requires constructing a similarity matrix ($O(n^2)$ complexity).

A comparison of the clustering algorithms

2.3. Clustering

Clustering of unlabeled data can be performed with the module sklearn.cluster. Each clustering algorithm comes in two variants: a class, that implements the fit method to learn the clusters on trai...

scikit-learn



Clustering is a challenging research field (1)

Clustering is a complex and active research area.
A good clustering algorithm must meet several key requirements to handle real-world data effectively.

1. Scalability
   - Must handle large datasets (millions or billions of objects).
   - Sampling can cause bias, so algorithms should be highly scalable in time and memory.
2. Handling Different Types of Attributes
   - Should work not only with numeric data, but also:
     - Categorical, binary, and ordinal data
     - Mixed data types
     - Complex data (e.g., images, text, graphs, sequences)
3. Discovery of Clusters with Arbitrary Shapes
   - Many methods assume spherical clusters (due to Euclidean distance).
   - Real-world clusters may have arbitrary forms (e.g., forest fire boundaries, sensor data).
   - Algorithms must detect non-spherical, unevenly sized, and irregular clusters.
4. Minimal Domain Knowledge (Input Parameters)
   - Many methods require user-defined parameters (e.g., number of clusters).
   - Hard to estimate for high-dimensional or unfamiliar data.
   - Algorithms should reduce dependency on manual parameter tuning.

Clustering is a challenging research field (2)

5. Robustness to Noisy Data
   - Real-world data often contains outliers, missing, or erroneous values.
   - Clustering algorithms should be robust to such noise to maintain cluster quality.
6. Incremental and Order-Insensitive Processing
   - Should allow incremental updates (new data added without full recomputation).
   - Results must be independent of data input order.
7. Capability for High-Dimensional Data
   - Modern datasets (e.g., documents, genomics) may have thousands of attributes.
   - Algorithms must effectively cluster sparse, high-dimensional data.
8. Constraint-Based Clustering
   - Must support real-world constraints (e.g., geographic, business, or capacity limits).
9. Interpretability and Usability
   - Clustering results must be:
     - Easy to interpret and explain
     - Useful for decision-making and business applications

Applications of Clustering Techniques (1)

1. Customer Segmentation (Marketing & Retail)
   - Goal: Group customers with similar purchasing behavior, interests, or demographics.
   - Why useful: Enables targeted marketing, personalized recommendations, and optimized pricing strategies.
   - Example:
     - E-commerce platforms cluster users by spending habits.
     - Banks cluster clients by risk and product usage patterns.
2. Image Segmentation and Computer Vision
   - Goal: Divide an image into meaningful regions (e.g., background, objects).
   - Why useful: Simplifies image analysis and helps in object recognition or medical image interpretation.
   - Example:
     - Medical imaging: segmenting tumors in MRI scans.
     - Autonomous vehicles: separating roads, pedestrians, and vehicles.
3. Document and Text Clustering (NLP)
   - Goal: Organize large collections of documents or articles based on their content similarity.
   - Why useful: Improves search results, topic discovery, and information retrieval.
   - Example:
     - Grouping news articles by topic.
     - Clustering research papers by field of study.

Applications of Clustering Techniques (2)

4. Anomaly and Fraud Detection
   - Goal: Identify unusual patterns or data points that do not fit any cluster.
   - Why useful: Detects fraudulent transactions, cyber intrusions, or faulty sensor readings.
   - Example:
     - Banks detecting abnormal credit card usage patterns.
     - Network security systems detecting suspicious activity.
5. Social Network and Community Detection
   - Goal: Find groups of people or entities that interact more frequently with each other than with outsiders.
   - Why useful: Helps understand social structures and influence networks.
   - Example:
     - Identifying communities in online social platforms (e.g., Twitter, Facebook).
     - Detecting collaboration networks among researchers.
6. Biology and Bioinformatics
   - Goal: Discover patterns in biological or genetic data.
   - Why useful: Helps understand species relationships, disease subtypes, or gene expression profiles.
   - Example:
     - Grouping genes with similar expression in microarray data.
     - Classifying cell types in single-cell RNA sequencing.

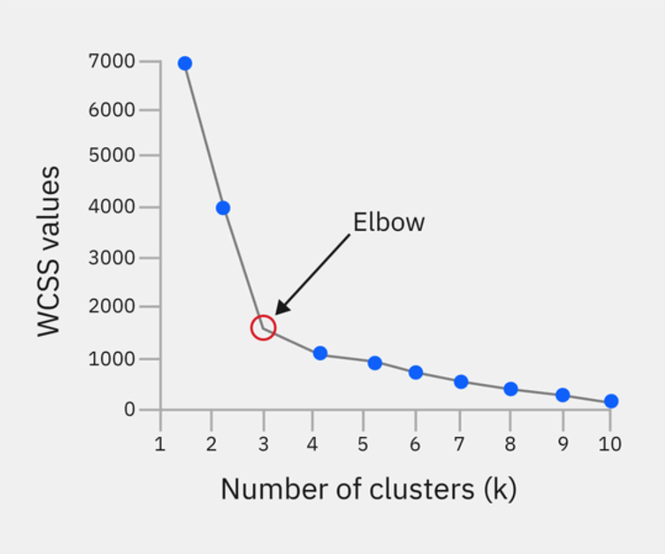
Applications of Clustering Techniques (3)

7. Recommender Systems
   - Goal: Cluster users or items based on similarity to provide better recommendations.
   - Why useful: Reduces complexity and improves accuracy of user–item matching.
   - Example:
     - Netflix clusters users with similar viewing patterns.
     - Spotify clusters songs by audio features to recommend new music.
8. Geographic and Spatial Analysis
   - Goal: Identify regions with similar characteristics or density patterns.
   - Why useful: Supports urban planning, traffic management, and environmental studies.
   - Example:
     - Grouping regions by air pollution levels.
     - Finding clusters of earthquakes or crime incidents.
9. Healthcare and Medical Research
   - Goal: Segment patients based on symptoms, genetics, or treatment responses.
   - Why useful: Enables personalized medicine and early disease detection.
   - Example:
     - Clustering diabetic patients based on blood sugar patterns.
     - Grouping patients by response to a particular drug.
10. Data Compression and Dimensionality Reduction
   - Goal: Replace large datasets with representative cluster centroids.
   - Why useful: Speeds up computation and reduces memory use.
   - Example:
     - Image compression by clustering similar pixels (K-Means).
     - Summarizing sensor data in IoT systems.

Finding the Optimal Number of Clusters (k) in K-Means

Why It Matters

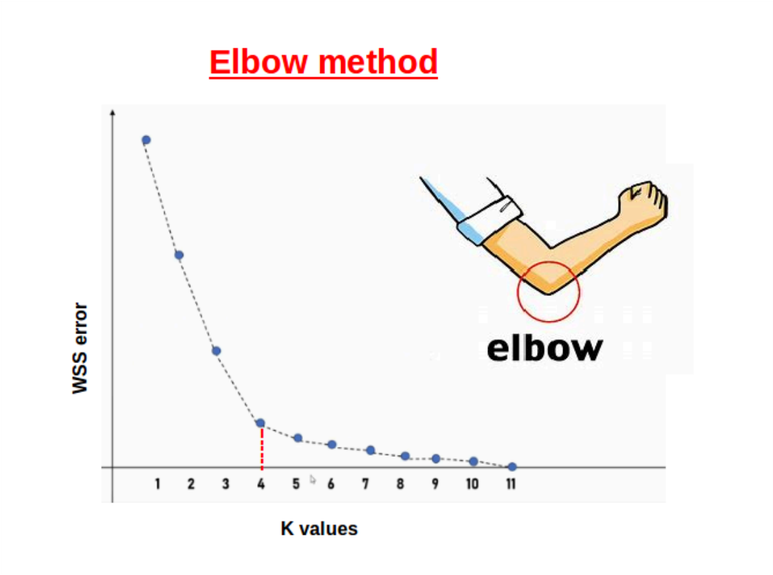
- Too few clusters → different groups get merged (loss of structure).
- Too many clusters → overfitting (noise treated as signal).
- Goal: choose k that balances compactness and separation.



a) Elbow Method

Idea: Plot how the total within-cluster variance (inertia) changes with k.

- Compute SSE (Sum of Squared Errors) for different k.
- Plot k vs. SSE.
- Look for the "elbow point" — where improvement slows sharply.



Lower SSE → points are closer to their centroid → better cohesion.

However, SSE always decreases as k increases (because more clusters = smaller groups).

That's why we use the Elbow Method: find the point where SSE reduction starts to slow down.

Inertia (also called SSE — Sum of Squared Errors)

For k clusters $C_1, C_2, \ldots, C_k$:

$$\text{SSE (Inertia)} = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||^2$$

Where:

- x = data point
- μ_i = centroid (mean vector) of cluster i
- $||x - \mu_i||^2$ = squared Euclidean distance between point and its centroid

Finding the Optimal Number of Clusters (k) in K-Means

(**b) Silhouette Method**
Idea: Measures how similar an object is to its own cluster vs. others.

- Score range: –1 → +1
- Higher = better defined clusters.

Choose k with the highest average silhouette score.

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Where:

- $a(i)$ = average distance of point *i* to points in the same cluster
- $b(i)$ = average distance of point *i* to points in the nearest other cluster

Interpretation:

- $s(i)$ close to **+1** → well-clustered
- $s(i)$ near **0** → between clusters
- $s(i)$ close to **–1** → probably in the wrong cluster

Finding the Optimal Number of Clusters (k) in K-Means

(c) Domain Knowledge

Sometimes the optimal k is guided by:

- Business rules (e.g., 4 customer tiers).
- Operational constraints (e.g., 5 managers, 10 regions).
- Always combine quantitative + contextual reasoning.

Thank You!